Checking the Functional Safety of Multi-ECU Over-the-Air Update Campaigns: A TLA$^+$ Model, a Pair Abstraction and Software-in-the-Loop Validation

Sravasti Bose

***Abstract*—Over-the-air (OTA) updates in software-defined vehicles increasingly change several electronic control units (ECUs) in one campaign. If ECUs depend on one another's versions, a partial failure can leave a vehicle running an incompatible mix, even when every ECU recovers correctly on its own. We ask whether such campaign-level safety can be checked formally, and whether the model's predictions match an executable system. We model a dual-bank, coordinator-driven campaign in TLA$^+$ and define four safety properties (compatibility at all times, a retained way back, all-or-nothing at campaign end, only verified images run) and one liveness property (the campaign finishes). The TLC model checker confirms them on a three-ECU campaign with up to two faults, and finds counterexamples in three flawed designs. One of them, rolling back only the failed ECU, violates compatibility after a single fault and passes every property without faults. Direct checking on one laptop finished for all generated campaigns up to 8 ECUs and for none at 10. A pair abstraction reduces safety checking for any number of ECUs to two models of 197 and 148 states, by a hand-written proof supported by mechanical refinement checks. In a software-in-the-loop simulator with separate ECU processes, file-backed flash and process kills as power loss, 6,360 fault-injected runs gave 0 misses and 0 false alarms against predictions frozen beforehand. Because model and simulator share an author and a schedule, this is evidence of consistency rather than independent validation. Hardware validation is future work.**

***Index Terms*—Over-the-air update, software-defined vehicle, multi-ECU campaign, rollback, model checking, TLA$^+$, functional safety.**

I. Introduction

OVER-THE-AIR updates let a manufacturer change vehicle software after sale. When one feature spans several ECUs, the update becomes a *campaign*: several ECUs must move to new versions that are only compatible with each other. Each ECU can protect itself with a dual-bank (A/B) layout, keeping its old image until the new one is confirmed. That protection is local. A campaign can still leave the vehicle in a state that no single ECU considers wrong, for example a brake controller running a version that needs a gateway version which has just been rolled back.

Regulation already asks for this. UN Regulation No. 156 requires manufacturers to identify interdependencies of the updated system with other systems, to confirm compatibility with the target vehicle's configuration before an update is issued, and to ensure the vehicle can restore its previous version, or reach a safe state, after a failed or interrupted update [1]. ISO 24089 sets requirements for software update engineering, including for ECUs, but does not prescribe technologies [2]. Neither says how to show that a particular campaign design meets these goals.

Formal work on vehicle updates has so far concentrated on security. Uptane [3], [4], which builds on TUF [5], has been analysed with CSP [6], [7], Tamarin [8], and a combination of the Kind 2 model checker with Tamarin, which found six new vulnerabilities [9]. UniSUF has been verified in ProVerif [10]. These analyses ask what an attacker can do. Uptane's designers also list a "mixed-bundles" attack, in which ECUs are made to install versions that must not run together, and note that partial installation can also happen by accident, for example when power runs out [3]. We study that hazard when no attacker is involved, and it is the update design itself, under faults, that leaves the vehicle in an incompatible or mixed state.

We ask two questions. *Can functional safety of a multi-ECU campaign, including version dependencies, partial failure and rollback, be checked formally? And do the model's predictions match the behaviour of an executable implementation?*

Our contributions, each tied to one result:

1) A TLA$^+$ model of a coordinator-driven A/B campaign, with four safety properties and one liveness property stated in plain language first (Section IV, Table II).
2) A model-checking study that confirms the properties for a correct design and finds counterexamples in three flawed designs, including a rollback flaw invisible without faults (Section V, Table IV).
3) A measurement of where direct checking stops being practical, and a pair abstraction that covers safety for any number of ECUs, with a written proof and mechanical checks (Section VI, Table VI, Fig. 2).
4) A software-in-the-loop (SIL) comparison of frozen model predictions against fault-injected runs (Section VII, Tables XII and XIII).

All code, raw results and run manifests are in the artifact repository https://github.com/sravasti12/OTA-Campaign-Verfier-Research-Artifact. Every number in this paper is generated from those raw files by a script.

S. Bose is an independent researcher, Kolkata, India (e-mail: bosesravasti8@gmail.com).

Draft, not yet submitted.

II. Background

A/B updates. Each ECU has two slots. It runs from one while the new image is written to the other, verifies it, switches to it for a trial, and confirms it if healthy. Until confirmation, switching back restores the old version.

Campaigns. A coordinator sequences downloads, activations and confirmations across ECUs. A dependency "x requires d" means version 2 of x may run only while d runs version 2. On failure the coordinator rolls the campaign back.

TLA⁺ and TLC. TLA⁺ specifies a system as an initial-state predicate and a next-state relation, with temporal formulas for fairness and liveness [11]. TLC is an explicit-state model checker for TLA⁺ [12]. It enumerates reachable states, checks invariants and temporal properties, and returns a shortest counterexample for a violated invariant. We use release v1.7.4 (TLC 2.19) [13]. Apalache is a symbolic alternative [14], and TLAPS a proof system for TLA⁺ [15]. We used neither. TLA⁺ has been used on production distributed systems in industry [16].

III. Related Work

Security of update frameworks. Table I lists the formal analyses of vehicle software updates that we located. TUF [5] and Uptane [3], [4] protect updates against compromised repositories and keys. Their analyses [6]–[9], [19], [20] and that of UniSUF [10] all fix an attacker and ask what it can achieve. "Rollback" in that literature means a downgrade attack, in which an ECU is made to install an older, vulnerable image [3], [9]. In this paper it means recovery after a failure. Hagen et al. name extension to multi-ECU and large-scale vehicular networks as future work, and point to assume–guarantee reasoning against state explosion [10]. Uptane's director repository performs dependency resolution when choosing which images a vehicle installs [4]. That concerns which versions are *selected*. Our properties concern which versions *run* while a campaign executes and after it fails part way.

Safety and the update process. Pedroza et al. extended a safety-oriented modelling framework, which verifies safety properties with UPPAAL, to also cover security, and applied it to a firmware update protocol for a single in-car ECU [17], [18]. Model checking has also been used for functional safety of automotive runtime logic, such as the arbitration of a fail-operational driving system, as evidence for ISO 26262 [23]. Implementation-level work includes ScalOTA, a scalable OTA architecture with a formalized chain of trust [24], and an AUTOSAR-based firmware OTA implementation on a steering system [25]. Surveys cover OTA security for connected vehicles [26] and the software-defined vehicle more broadly [27].

Method. Our pair abstraction is an over-approximating, safety-preserving abstraction in the sense of Clarke et al. [28]. It checks a component against a stand-in environment, in the spirit of compositional model checking [29]. Its refinement check, which maps full states to pair states, is a refinement mapping [30]. Results showing that a small, fixed instance covers a whole family are known as cutoffs [31]. Our proof is direct and uses none of those theorems. Trace validation links TLA⁺ specifications to implementations [32], [33], a route to strengthening our SIL comparison.

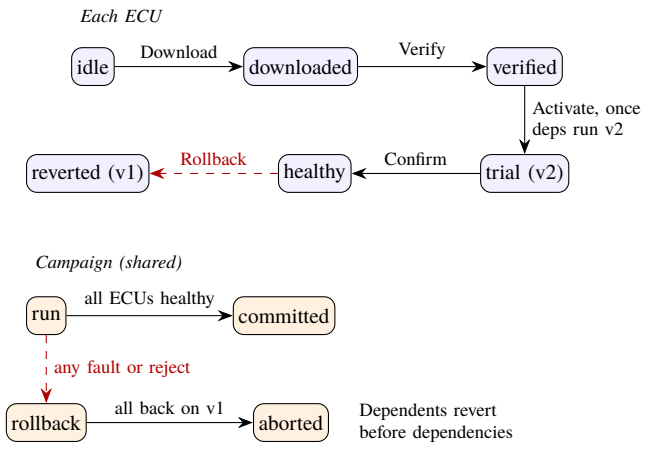


Fig. 1. The campaign model. Top: one ECU's stages. Bottom: the shared campaign state. Dashed: a fault and the rollback it starts. Faults are download failure, corrupt download, trial failure and vehicle-wide power loss, and each uses one unit of the budget F.

Gap. In the sources we searched, none model-checks version compatibility across dependent ECUs under partial failure and recovery, or compares such a model against an executable implementation. We searched Crossref, arXiv, OpenAlex and the open web, and followed the reference lists of [9], [10]. We did not search the subscription databases IEEE Xplore and Scopus.

IV. Campaign Model and Properties

A. Model

The model, `OTACampaign.tla`, is parameterized by a set of ECUs, an acyclic dependency relation D, a fault budget F and a design flag. Each ECU has an active version (1 or 2), the version held in its other slot, a stage, and a corruption flag. A shared variable records the campaign as *run*, *rollback*, *committed* or *aborted*. Fig. 1 shows the steps.

Correct design. An ECU activates only after the ECUs it depends on run version 2. On any failure the campaign enters rollback, and every ECU on version 2 reverts, dependents before their dependencies. The campaign then ends as aborted.

Faults. Download failure, corrupt download, trial failure and vehicle-wide power loss, which discards unverified downloads and starts rollback. Power loss may also strike during rollback, which then resumes.

Flawed designs. One flag changes one decision. `LocalRollback` reverts only the ECU that failed. `EarlyErase` discards the old image on activation to free flash. `SkipVerify` accepts any downloaded image.

Simplifications. Steps are atomic. The campaign state is one durable, shared variable, so there is no message loss or delay between coordinator and ECUs. There are no timeouts or retries. Each ECU has two versions. These limits recur in Section VIII.

TABLE I
FORMAL ANALYSES OF VEHICLE SOFTWARE UPDATES THAT WE LOCATED, AND THE OBJECT OF THIS PAPER

Work	Object	Technique	Properties
Pedroza et al. [17]	Firmware update protocol between a communication unit and one ECU [18]	AVATAR/SysML; ProVerif, with UPPAAL for safety	Security of the protocol (authenticity, confidentiality reported); framework also supports safety
Mahmood et al. [19], [20]	Uptane implementation	Threat assessment; attack-tree-based test generation	Security, by testing
Kirk et al. [6], [7]	Uptane	CSP models verified to generate tests	Security against a Dolev–Yao attacker
Mukherjee et al. [21]	Uptane inside a TEE on COTS hardware	SAW (SAWScript)	Functional and logical inconsistencies in the code
Ponsard and Darquennes [22]	UpKit IoT update protocol	Tamarin	Firmware integrity, freshness of requests*
Boureanu [8]	Uptane 2.0	Tamarin	Security and privacy
Lorch et al. [9]	Uptane	Kind 2 with Tamarin	Security under nine threat models, including rollback (downgrade) attacks
Hagen et al. [10]	UniSUF	ProVerif	Confidentiality, integrity, authenticity, freshness, order; liveness argued
This paper	Multi-ECU A/B campaign with version dependencies	TLA$^+$/TLC, pair abstraction, SIL	Compatibility, retained way back, all-or-nothing at end, verified images only, termination, under faults and recovery

*As described by Hagen et al. [10]. We have read only the record of this paper, not its full text.

TABLE II
PROPERTIES CHECKED

	Formal statement	What it does *not* claim
S1	For every $(x, d) \in D$: x runs v2 $\Rightarrow$ d runs v2, in every reachable state	Version ranges; dependencies among three or more ECUs
S2	Before commit, every ECU running v2 holds v1 in its other slot	That the kept image boots
S3	Once committed or aborted, all ECUs run v1 or all run v2	Consistency at intermediate states
S4	No ECU runs an image flagged corrupt	Any security property
L1	Under weak fairness and at most F faults, the campaign reaches committed or aborted	Termination with unbounded faults or timeouts

TABLE III
STATE SPACE OF THE THREE-ECU CAMPAIGN: DISTINCT / GENERATED STATES, FROM A SAFETY PROPERTY THAT HOLDS (SO THE SPACE IS EXPLORED COMPLETELY)

	0 faults		1 fault		2 faults	
Design	Dist.	Gen.	Dist.	Gen.	Dist.	Gen.
Correct	96	218	674	1,077	1,224	2,080
LocalRollback	96	218	649	952	1,174	1,723
EarlyErase	96	218	437	696	746	1,151
SkipVerify	96	277	724	1,498	2,095	3,929

TABLE IV
VERDICTS ON THE THREE-ECU CAMPAIGN AT FAULT BUDGET 2. PARENTHESES: SMALLEST BUDGET AT WHICH THE VIOLATION APPEARS

Design	S1	S2	S3	S4	L1
Correct	holds	holds	holds	holds	holds
LocalRollback	**violated** (1)	holds	**violated** (1)	holds	holds
EarlyErase	holds	**violated** (0)	holds	holds	**violated** (1)
SkipVerify	holds	holds	holds	**violated** (1)	holds

B. Properties

The properties were written in plain English before the model. Table II gives each one and the wording the results support.

V. MODEL CHECKING THE DESIGNS

We checked a three-ECU campaign (gateway, brake, infotainment; brake requires gateway) for all four designs and five properties at fault budgets 0, 1 and 2, plus three sanity checks: TLC was asked to find a commit, an abort and a real rollback in the correct design, so that a pass cannot come from a model that does nothing. All 63 checks matched verdicts recorded before the first run (at budget 2). The correct design was fully explored with 96, 674 and 1,224 distinct states at budgets 0, 1 and 2. Table IV gives the verdicts, and Table III the size of every design's state space. All designs coincide without faults, apart from TLC counting `SkipVerify`'s verification step twice. With faults, `SkipVerify` has the largest space, because a corrupt download continues instead of starting rollback. The other flawed designs have smaller spaces than the correct one.

Table V lists the counterexamples. Each one was replayed by an independent checker (all replayed: yes).

Rolling back only the failed ECU. Gateway and brake both activate version 2. Gateway then fails its health check, and only gateway reverts. Brake keeps running version 2 against a gateway on version 1, violating S1, and the campaign ends as aborted with the mix in place. With no faults this design passes every property, so nominal testing would not reveal it.

Erasing early. With no faults at all, an ECU that activates version 2 has no way back, violating S2. With one corrupt download detected late, two ECUs cannot revert, and the campaign stays in rollback forever (L1). This design "holds" S3 only because the stuck campaign never ends, so S3 and L1

TABLE V
SHORTEST COUNTEREXAMPLES, AT THE SMALLEST FAULT BUDGET THAT PRODUCES EACH VIOLATION

Design	Prop.	Faults	States	Ending	Explored	Replayed
LocalRollback	S1	1	9	finite	406	yes
LocalRollback	S3	1	6	finite	158	yes
EarlyErase	S2	0	4	finite	11	yes
EarlyErase	L1	1	11	stuttering	437	yes
SkipVerify	S4	1	4	finite	56	yes

States: length of the trace. Ending: *stuttering* means the trace ends in a state the campaign never leaves. Explored: distinct states TLC visited before stopping. Replayed: the trace was re-executed step by step by a separate Python checker that does not use TLC.

TABLE VI
DIRECT CHECKING OF S1–S4, FAULT BUDGET 1

ECUs	Finished	Limit (s)	Median states	Median time (s)	Peak MB
5	3 of 3	1,200	23,979	4.94	365
6	3 of 3	1,200	47,120	6.58	629
7	3 of 3	1,200	275,083	13.10	1,540
8	3 of 3	1,200	4,500,928	274.89	1,914
9	2 of 3	1,200	9,050,115†	662.59	2,218
10	0 of 3	1,200	–	–	2,189
20	0 of 3	300	–	–	1,686
50	0 of 3	300	–	–	1,540

†Finished runs only, so the value understates the true median. Runs that stopped report no totals.

are needed together.

Skipping verification. A corrupt image runs after one fault (S4). This variant exists to show that S4 can fail.

Every property was violated by at least one design, which shows that each one can fail in this model.

VI. SCALABILITY AND ABSTRACTION

A. Direct checking

We generated acyclic dependency graphs with a seeded generator, 3 graphs per size, and checked S1–S4 together with TLC at a fault budget of 1 (1 worker, 4 GB heap) on one laptop with 8 logical CPUs and Java 18.0.2. Sizes up to 10 ran with a limit of 1,200 s, and sizes 20 and 50 with 300 s. We call checking *practical* when TLC finishes within the limit. This is our definition, tied to this machine.

All graphs finished up to 8 ECUs, 2 of 3 at 9, and none at 10 or above (Table VI, Fig. 2). Time, not memory, was the limit: peak memory stayed at or below 2,218 MB. With a five-minute budget the boundary would be 8 ECUs, as the slowest run there took 294.85 s. The graph mattered as much as the size. At 7 ECUs, a graph with 1 edge reached 1,172,561 states and one with 6 edges 190,288, because fewer dependencies allow more interleavings. Three graphs per size are too few to fit a growth rate, so we report none. The liveness property L1 was checked directly only at 3 to 6 ECUs, and held in all 12 runs (Table VII).

B. Pair abstraction

Each safety property mentions at most two ECUs. We therefore check two ECUs $\{a, b\}$ against an environment that may fail at any time. The pair model keeps the two ECUs' variables and the campaign state, drops the fault counter, and adds one action, *EnvFail*, which moves the campaign from run to rollback. An abstraction function α restricts a full state to the pair.

TABLE VII
DIRECT CHECKING OF LIVENESS L1 (CORRECT DESIGN, FAULT BUDGET 1). VALUES PER GRAPH, SEEDS 1 / 2 / 3

ECUs	Edges	Distinct states	Max time (s)	L1 holds
3	1 / 1 / 2	674 / 674 / 529	1.79	3 of 3
4	2 / 4 / 1	3,405 / 1,694 / 4,358	2.34	3 of 3
5	0 / 2 / 4	39,379 / 23,979 / 12,075	6.43	3 of 3
6	7 / 5 / 5	47,120 / 36,025 / 76,484	13.71	3 of 3

Theorem 1. Fix a design. If S1–S4 hold in both pair shapes (no edge, and one edge a requires b), then S1–S4 hold in every reachable state of the full model for every finite set of at least two ECUs, every acyclic D and every fault budget.

Proof sketch. α maps the initial state to the pair's initial state. Every full step maps to a pair step or a stutter. Steps of ECUs outside the pair either leave the pair unchanged or start rollback, matched by *EnvFail*. Steps inside the pair have the same effect and weaker guards in the pair model, since the pair sees only its own dependencies and has no fault budget. By induction, α of every reachable full state is reachable in a pair model. Renaming reduces every pair to the two shapes. S1, S2 and S4 then transfer directly. For S3, equal versions on every pair imply equal versions on all ECUs. The full proof is in the artifact. It has *not* been machine-checked, for example in TLAPS [15], or independently reviewed, and we present it as a proof sketch.

The abstraction over-approximates [28], so a pair "violated" verdict is only a warning, while "holds" is a guarantee. It says nothing about L1. For L1 we give only an informal argument, which needs acyclic D, and treat it as such.

Cost. The correct design's two pair models have 197 and 148 distinct states and took 1.93 s and 3.57 s, independent of the number of ECUs. At 9 ECUs, the median finished direct run had about 45,940 times as many states as the larger pair model. Table VIII gives the pair models for every design. For each design, the pair models flag exactly the safety properties that the three-ECU campaign violates (Table IV). L1 is outside the pair models, and `LocalRollback`'s S1 warning appears only in the shape with an edge, as S1 constrains only dependent pairs.

Mechanical checks. These test the proof's lemmas on finite instances. They support the proof but do not replace it. (i) Refinement: TLC checked that the full model refines the pair model through α for every ECU pair, in 36 of 36 runs on small generated campaigns over all four designs (Table X). (ii) Negative control: with *EnvFail* disabled, the same check was violated, so the check can detect a broken abstraction. (iii) Agreement: on 3, 4, 5 ECUs at a fault budget of 2, pair predictions matched full-model verdicts in all 144 comparisons (109 both hold, 35 both violated), with 0 unsound and 0 spurious results (Table IX). That zero applies only to these

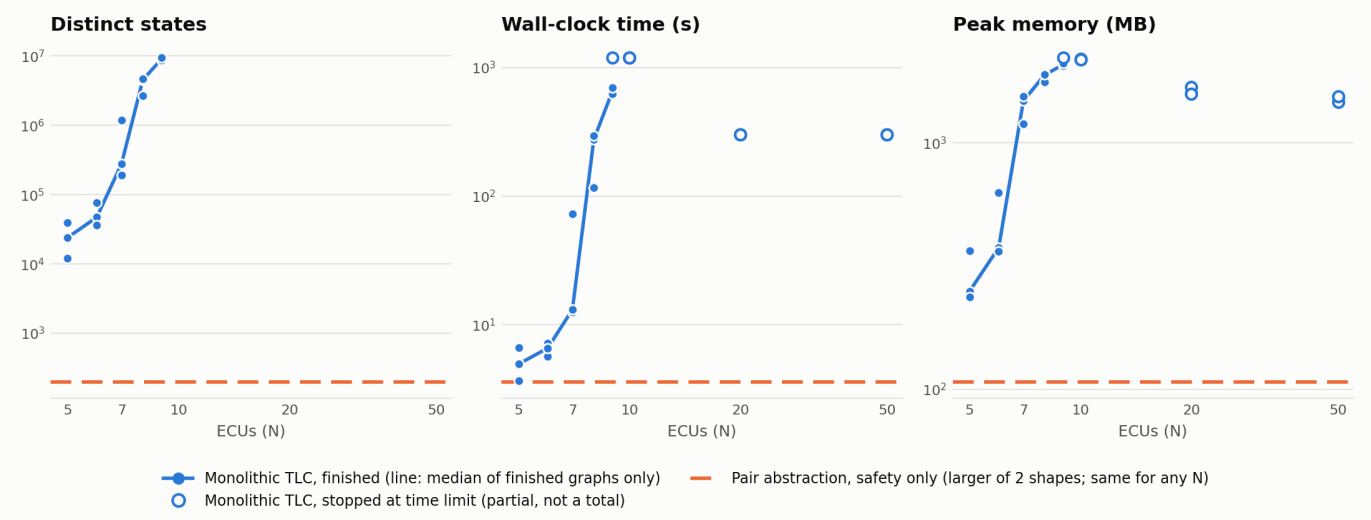


Fig. 2. Direct checking of S1–S4 against the number of ECUs (log-log). Hollow markers: runs stopped at the time limit, whose values are not totals. Dashed: the larger of the two pair models, whose cost does not depend on the number of ECUs.

TABLE VIII
PAIR MODELS: DISTINCT STATES (FROM A PROPERTY THAT HOLDS) AND PROPERTIES VIOLATED

Design	Shape	States	Violated
Correct	none	197	–
Correct	edge	148	–
LocalRollback	none	193	S3
LocalRollback	edge	149	S1 S3
EarlyErase	none	141	S2
EarlyErase	edge	117	S2
SkipVerify	none	407	S4
SkipVerify	edge	289	S4

TABLE IX
PAIR PREDICTION AGAINST FULL-MODEL VERDICT, 3, 4, 5 ECUS, FAULT BUDGET 2, S1–S4

Design	Both hold	Both violated	Unsound	Spurious
Correct	36	0	0	0
LocalRollback	19	17	0	0
EarlyErase	27	9	0	0
SkipVerify	27	9	0	0

TABLE X
REFINEMENT CHECKS OF THE FULL MODEL AGAINST THE PAIR MODEL, ALL FOUR DESIGNS, THREE GRAPHS PER SIZE, FAULT BUDGET 1

ECUs	Runs	Pairs/run	Distinct states	Max time (s)	Holds
3	12	3	366–724	1.93	12 of 12
4	12	6	1,237–4,642	4.15	12 of 12
5	12	10	7,207–40,999	32.81	12 of 12

TABLE XI
SIL CAMPAIGNS. RUNS = SCRIPTS × 3 SCHEDULES × 4 DESIGNS

Campaign	ECUs	Edges	Runs	Paths	Viol.	Mismatch
Three-ECU	3	1	912	597	321	0
5 ECUs, seed 1	5	0	1,488	989	572	0
5 ECUs, seed 2	5	2	1,488	989	578	0
5 ECUs, seed 3	5	4	1,488	989	578	0
10 ECUs, seed 1	10	9	984	864	426	0

Paths: distinct predicted action sequences per design and schedule. Viol.: property cells where a violation was predicted and observed. Mismatch: misses plus false alarms.

instances.

VII. VALIDATION AGAINST A SOFTWARE-IN-THE-LOOP SIMULATOR

A. Setup

The SIL runs each ECU as a separate operating-system process with file-backed slots and a durable boot record. Power loss kills every ECU process at once and restarts them from disk. Verification compares a SHA-256 digest of the image, and a corrupt download really corrupts bytes. The coordinator keeps its campaign state in a durable file and is rebuilt from it after power loss. An oracle judges S1–S4 from the bytes on disk after every step, not from what the coordinator believes. L1 is judged in finite runs as "the campaign has ended". The SIL was written from the plain-language design and imports nothing from the model or its tooling. A test enforces this.

Campaigns and faults. The SIL runs 5 campaigns (Table XI): the three-ECU campaign, three generated 5-ECU campaigns and one 10-ECU campaign. Each runs under three schedules (batch, rolling, and late confirmation, where trial periods overlap) and all four designs. Fault scripts cover no fault, every applicable single fault at every step, and, for campaigns of at most five ECUs, each single fault followed by a power loss one or three steps later. For the three-ECU campaign this gives 76 scripts per schedule and design.

Predictions. A scripted version of the model, pinned to the same schedule and fault script, predicted for each run the violated properties, the action path and the end state. The predictions were generated from commit `19423e3` and

TABLE XII
MODEL PREDICTIONS AGAINST SIL OBSERVATIONS, PER PROPERTY. ALL RUNS (DISTINCT RUNS IN PARENTHESES)

	Match (viol.)	Match (safe)	Miss	False alarm
S1	20 (15)	4,852 (3,424)	0 (0)	0 (0)
S2	846 (577)	5,514 (3,851)	0 (0)	0 (0)
S3	676 (453)	5,684 (3,975)	0 (0)	0 (0)
S4	102 (102)	6,258 (4,326)	0 (0)	0 (0)
L1	831 (562)	5,529 (3,866)	0 (0)	0 (0)

TABLE XIII
MODEL PREDICTIONS AGAINST SIL OBSERVATIONS, BY DESIGN AND BY SCHEDULE (ALL RUNS, ALL PROPERTIES)

	Match (viol.)	Match (safe)	Miss	False alarm
Correct	0	7,578	0	0
LocalRollback	696	6,882	0	0
EarlyErase	1,677	5,901	0	0
SkipVerify	102	7,476	0	0
Batch	570	9,534	0	0
Rolling	1,353	8,751	0	0
Late confirmation	552	9,552	0	0

committed as `e593539`, and the SIL then ran from that same commit.

B. Results

Each run and property is one cell: a violation predicted and observed (match), neither (match safe), observed but not predicted (miss), or predicted but not observed (false alarm). S1 cells in the campaign without edges are left out, because S1 cannot be violated there. Many fault scripts lead to the same run, so we also count distinct (campaign, design, schedule, path, verdict) combinations.

Over 6,360 runs, there were 2,475 matched violations, 27,837 matched safe cells, 0 misses and 0 false alarms (distinct runs: 1,709, 19,442, 0 and 0). The predicted action path matched the observed one in 6,360 runs and the end state in 6,360 (Table XII).

The correct design produced no violation in any run, and every flawed design produced violations that the model had predicted (Table XIII). All 20 S1 violations came from design `LocalRollback` under schedule "late confirmation". That is the only schedule in which a dependency can still be in its trial period, and so able to fail, after its dependent has activated. This is why `LocalRollback`'s S1 flaw needs a particular schedule to show.

Table XIV groups the runs by the first fault injected. The correct design violated nothing under any fault kind. `SkipVerify` failed only after a corrupt download, which is the one fault its flaw concerns. `EarlyErase` failed even with no fault, as the model predicts (Table IV). Each design column counts runs out of a quarter of the Runs column, since every script runs once per design.

How to read this. Agreement is the expected outcome when two implementations of one design follow the same schedule. The comparison shows that the model and an executable, process-level implementation agree on these campaigns and faults, including after real process kills and real hash checks. It does not show that the model captures failure modes the SIL cannot produce, such as torn writes, because the SIL injects only the model's fault kinds. A power cut during a write is the one fault that is lower-level in the SIL than in the model, where it maps to a power loss at that step.

TABLE XIV
SIL RUNS BY THE FIRST FAULT INJECTED, AND HOW MANY RUNS OF EACH DESIGN VIOLATED AT LEAST ONE PROPERTY (AS OBSERVED ON DISK, AND AS PREDICTED)

First fault	Runs	Correct	LocalRb.	EarlyErase	SkipVer.
No fault	60	0	0	15	0
Power loss	1,404	0	161	214	0
Download failure	336	0	23	23	0
Corrupt download	336	0	23	23	84
Power cut mid-write	336	0	23	23	0
Trial failure	336	0	74	84	0
Power loss + later power loss	1,824	0	196	272	0
Download failure + later power loss	432	0	28	28	0
Corrupt download + later power loss	432	0	28	28	18
Power cut mid-write + later power loss	432	0	28	28	0
Trial failure + later power loss	432	0	92	108	0

VIII. DISCUSSION AND THREATS TO VALIDITY

Construct. The properties are narrower than "safe update". S3 is checked only at campaign end, S2 does not establish that the kept image boots, and S4 is not a security property (Table II). Each ECU has two versions, and dependencies are pairwise.

Model scope. Steps are atomic, and the campaign state is shared and durable, with no message loss, delay, timeouts or retries. The pair-abstraction proof depends on the shared campaign variable, and message loss would add behaviours it does not cover.

Internal. The same author wrote the model, the flawed designs, the SIL and the fault scripts, and the flaws were chosen, not found. Agreement between model and SIL is therefore weak evidence of the model's accuracy. An independent update client and fault kinds outside the model, such as torn writes and message loss, would strengthen it. We leave both to future work.

Proof. Theorem 1 is a hand proof, checked by us and by finite mechanical tests only. L1 beyond 6 ECUs rests on an argument, not a proof.

Measurement. Each scaling configuration ran once on one Windows laptop, so timings have no variance estimate. Our other work overlapped the timings at 5 and 6 ECUs, though not their state counts. Limits of 300 s at 20 and 50 ECUs show only that TLC did not finish in that time. A server, longer limits, multi-worker TLC or Apalache [14] would move the boundary. The fault budget was 1 in the scaling runs.

External. Nothing here was run on vehicle hardware, real ECU flash, CAN, or AUTOSAR software. A Raspberry Pi testbed with relay-switched power cuts is designed but not built, and hardware results are future work.

Standards. UN Regulation No. 156 asks that interdependencies be identified, compatibility be confirmed before

release, and the vehicle be able to return to its previous version or a safe state after a failed or interrupted update [1]. Our properties S1–S3 are one way to make the last of these precise for a campaign. We draw no conclusion about compliance or certification, which concern a manufacturer's processes as well as its designs. We have read only the scope of ISO 24089 [2], not its requirements.

Literature search. The related-work search did not cover IEEE Xplore or Scopus (Section III), so relevant work indexed only there may be missing.

IX. Conclusion

A dual-bank layout makes each ECU recoverable, but not the vehicle. A small TLA$^+$ model with five properties separates a correct campaign from three plausible mistakes. One of them, rolling back only the failed ECU, is invisible without faults. Direct model checking reached 8 ECUs on a laptop. A pair abstraction moves safety checking to two small models for any number of ECUs, resting on a hand proof that still needs independent review. A process-level simulator agreed with frozen model predictions in every run we tried, which shows consistency, not independent validation. Next steps are independent fault sources, message-level coordination, and hardware power cuts.

Artifact

Model, scripts, raw results with manifests (commit, uncommitted files, seed, environment and SHA-256 of every output) and the generator for every number in this paper: https://github.com/sravasti12/OTA-Campaign-Verfier-Research-Artifact.

Appendix A
Every Direct Scaling Run

Table XV lists each run summarized in Table VI. In all 14 finished runs, the depth of the breadth-first search was exactly $4N+3$ for N ECUs (checked by the generator: yes), consistent with each ECU contributing its four nominal steps (download, verify, activate, confirm). The graph changes how many states there are, not how deep the search goes.

TABLE XV
Direct checking of S1–S4, every graph, fault budget 1

N	Seed	Edges	Status	Distinct	Generated	Depth	Time (s)	MB
5	1	0	completed	39,379	89,500	23	6.65	365
5	2	2	completed	23,979	51,277	23	4.94	250
5	3	4	completed	12,075	23,535	23	3.67	237
6	1	7	completed	47,120	98,858	27	7.21	376
6	2	5	completed	36,025	73,735	27	5.66	362
6	3	5	completed	76,484	168,947	27	6.58	629
7	1	6	completed	190,288	425,188	31	12.56	1,190
7	2	5	completed	275,083	643,330	31	13.10	1,479
7	3	1	completed	1,172,561	3,197,163	31	73.03	1,540
8	1	3	completed	4,500,928	12,808,278	35	274.89	1,914
8	2	4	completed	2,643,768	7,078,380	35	116.61	1,762
8	3	3	completed	4,648,212	13,227,072	35	294.85	1,887
9	1	7	completed	8,598,623	23,830,181	39	622.77	2,050
9	2	7	completed	9,501,607	26,485,445	39	702.40	2,102
9	3	4	stopped (time)	–	–	–	1,200.24	2,218
10	1	9	stopped (time)	–	–	–	1,200.27	2,189
10	2	5	stopped (time)	–	–	–	1,200.29	2,177
10	3	6	stopped (time)	–	–	–	1,200.29	2,175
20	1	14	stopped (time)	–	–	–	300.18	1,590
20	2	11	stopped (time)	–	–	–	300.14	1,686
20	3	19	stopped (time)	–	–	–	300.14	1,577
50	1	27	stopped (time)	–	–	–	300.11	1,532
50	2	32	stopped (time)	–	–	–	300.08	1,465
50	3	41	stopped (time)	–	–	–	300.10	1,540

References

[1] United Nations Economic Commission for Europe, "UN regulation no 156 – uniform provisions concerning the approval of vehicles with regards to software update and software updates management system," Official Journal of the European Union, L 82, 9.3.2021, pp. 60–74. http://data.europa.eu/eli/reg/2021/388/oj, 2021.

[2] ISO, "ISO 24089:2023 road vehicles – software update engineering," https://www.iso.org/standard/77796.html, 2023, accessed for scope only.

[3] T. K. Kuppusamy, A. Brown, S. Awwad, D. McCoy, R. Bielawski, C. Mott, S. Lauzon, A. Weimerskirch, and J. Cappos, "Uptane: Securing software updates for automobiles," 14th escar Europe. https://uptane.org/papers/kuppusamy_escar_16.pdf, 2016.

[4] T. K. Kuppusamy, L. A. DeLong, and J. Cappos, "Uptane: Security and customizability of software updates for vehicles," *IEEE Vehicular Technology Magazine*, vol. 13, no. 1, pp. 66–73, 2018.

[5] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine, "Survivable key compromise in software update systems," in *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*. ACM, 2010, pp. 61–72.

[6] R. Kirk, H. N. Nguyen, J. Bryans, S. Shaikh, D. Evans, and D. Price, "Formalising UPTANE in CSP for security testing," in *IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. IEEE, 2021, pp. 816–824.

[7] R. Kirk, H. N. Nguyen, J. Bryans, S. A. Shaikh, and C. Wartnaby, "A formal framework for security testing of automotive over-the-air update systems," *Journal of Logical and Algebraic Methods in Programming*, vol. 130, p. 100812, 2023.

[8] I. Boureanu, "Formally verifying the security and privacy of an adopted standard for software-update in cars: Verifying Uptane 2.0," in *IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2023, pp. 1301–1306.

[9] R. Lorch, D. Larraz, C. Tinelli, and O. Chowdhury, "A comprehensive, automated security analysis of the Uptane automotive over-the-air update framework," in *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. ACM, 2024, pp. 594–612.

[10] M. S. Hagen, E. Lundqvist, A. Phu, Y. Wang, K. Strandberg, and E. M. Schiller, "Towards a formal verification of secure vehicle software updates," *Computers & Security*, vol. 161, p. 104751, 2026, preprint arXiv:2511.15479.

[11] L. Lamport, "The temporal logic of actions," *ACM Transactions on Programming Languages and Systems*, vol. 16, no. 3, pp. 872–923, 1994.

[12] Y. Yu, P. Manolios, and L. Lamport, "Model checking TLA+ specifications," in *Correct Hardware Design and Verification Methods (CHARME)*, ser. Lecture Notes in Computer Science, vol. 1703. Springer, 1999, pp. 54–66.

[13] "TLA+ tools, release v1.7.4 (TLC 2.19)," https://github.com/tlaplus/tlaplus, sHA-256 of tla2tools.jar recorded in tools/versions.json.

[14] I. Konnov, J. Kukovec, and T.-H. Tran, "TLA+ model checking made symbolic," *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 1–30, 2019.

[15] D. Cousineau, D. Doligez, L. Lamport, S. Merz, D. Ricketts, and H. Vanzetto, "TLA+ proofs," in *FM 2012: Formal Methods*, ser. Lecture Notes in Computer Science. Springer, 2012, pp. 147–154.

[16] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff, "How Amazon Web Services uses formal methods," *Communications of the ACM*, vol. 58, no. 4, pp. 66–73, 2015.

[17] G. Pedroza, M. S. Idrees, L. Apvrille, and Y. Roudier, "A formal methodology applied to secure over-the-air automotive applications," in *IEEE Vehicular Technology Conference (VTC Fall)*. IEEE, 2011, pp. 1–5.

[18] M. S. Idrees, H. Schweppe, Y. Roudier, M. Wolf, D. Scheuermann, and O. Henniger, "Secure automotive on-board protocols: A case of over-the-air firmware updates," in *Communication Technologies for Ve-*

hicles (Nets4Cars/Nets4Trains), ser. Lecture Notes in Computer Science. Springer, 2011, pp. 224–238.

[19] S. Mahmood, A. Fouillade, H. N. Nguyen, and S. A. Shaikh, "A model-based security testing approach for automotive over-the-air updates," in *IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2020, pp. 6–13.

[20] S. Mahmood, H. N. Nguyen, and S. A. Shaikh, "Systematic threat assessment and security testing of automotive over-the-air (OTA) updates," *Vehicular Communications*, vol. 35, p. 100468, 2022.

[21] A. Mukherjee, R. Gerdes, and T. Chantem, "Trusted verification of over-the-air (OTA) secure software updates on COTS embedded systems," in *Third International Workshop on Automotive and Autonomous Vehicle Security (AutoSec)*. Internet Society, 2021.

[22] C. Ponsard and D. Darquennes, "Towards formal security verification of over-the-air update protocol: Requirements, survey and UpKit case study," in *Proceedings of the 7th International Conference on Information Systems Security and Privacy (ICISSP)*. SCITEPRESS, 2021, pp. 800–808.

[23] T. Schmid, S. Schraufstetter, J. Fritzsch, D. Hellhake, G. Koelln, and S. Wagner, "Formal verification of a fail-operational automotive driving system," arXiv:2101.07307, 2021.

[24] A. Shoker, F. Alves, and P. Esteves-Verissimo, "ScalOTA: Scalable secure over-the-air software updates for vehicles," arXiv:2307.02032, 2023.

[25] M. A. Mostafa, M. K. Mohamed, and R. W. Ezzat, "Enhancing AUTOSAR-based firmware over-the-air updates in the automotive industry with a practical implementation on a steering system," arXiv:2503.05839, 2025.

[26] B. Li, W. Hu, L. Da, Y. Wu, X. Wang, Y. Li, and C. Yuan, "Over-the-air upgrading for enhancing security of intelligent connected vehicles: A survey," *Artificial Intelligence Review*, vol. 57, no. 11, p. 314, 2024.

[27] E. Liotou, D. Tzelalidou, and G. Christodoulou, "The rise of the software-defined vehicle: Architectures, enabling technologies, and future opportunities," arXiv:2605.30001, 2026.

[28] E. M. Clarke, O. Grumberg, and D. E. Long, "Model checking and abstraction," *ACM Transactions on Programming Languages and Systems*, vol. 16, no. 5, pp. 1512–1542, 1994.

[29] E. M. Clarke, D. E. Long, and K. L. McMillan, "Compositional model checking," in *Proceedings of the Fourth Annual Symposium on Logic in Computer Science (LICS)*. IEEE Computer Society Press, 1989, pp. 353–362.

[30] M. Abadi and L. Lamport, "The existence of refinement mappings," *Theoretical Computer Science*, vol. 82, no. 2, pp. 253–284, 1991.

[31] E. A. Emerson and K. S. Namjoshi, "Reasoning about rings," in *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 1995, pp. 85–94.

[32] H. Cirstea, M. A. Kuppe, B. Loillier, and S. Merz, "Validating traces of distributed programs against TLA+ specifications," arXiv:2404.16075, 2024.

[33] H. Howard, M. A. Kuppe, E. Ashton, A. Chamayou, and N. Crooks, "Smart casual verification of the Confidential Consortium Framework," arXiv:2406.17455, 2024.